Nonuniform NC1 projection hardness of SENS group word problems via nested-commutator programs

Laurent Bartholdi, Michael Figelius
Markus Lohrey, Armin Weiß

doi:10.4230/LIPIcs.CCC.2020.29; CC BY 3.0 Unported

Editorial scope. One independent nonuniform NC1 projection mechanism with full SENS witnesses grouped. Does not claim complete uniform ALOGTIME theorem or PSPACE corollaries.

Allowed standard prerequisites (editorial).

- Group operations, balanced Boolean NAND formulas and standard machine/circuit normal form. Formal standard finite symmetric generating set includes identity for padding. Original references[3,12,17,47] retained as standard prerequisites; full new nested-commutator program and both explicit group constructions retained.

Formal non-uniform conclusion

Since the proof for the uniform case is not difficult but tedious, in this sketch we primarily focus on the non-uniform version, which only gives us hardness via (non-uniform) AC^0-reductions instead of DLOGTIME-reductions. This gives us a non-uniform AC^0-reduction (more precisely, a projection reduction) of $L(M)$ to $\mathrm{WP}(G)$.

Standard generating sets and word problems

Recall that a group G with identity element 1 is finitely generated (f.g. for short) if there is a finite set $\Sigma \subseteq G$ such that every element of G can be written as a product of elements of Σ; this product can be formally written as a word from Σ^*. For technical reasons we assume that $1 \in \Sigma$ (which is needed for padding reasons) and that for every $a \in \Sigma$ also the inverse a^{-1} belongs to Σ; such a generating set Σ is called standard. We have a natural involution on Σ^* defined by $(a_1 \cdots a_n)^{-1} = a_n^{-1} \cdots a_1^{-1}$ for $a_i \in \Sigma$ (which is the same as forming inverses in the group). For words $u, v \in \Sigma^*$ we write $u =_G v$ if u and v are equal in G; sometimes we just say $u = v$ in G. The word problem for G, $\mathrm{WP}(G)$ for short, is the question whether $u =_G 1$ holds for a given word $u \in \Sigma^*$. In this formulation the word problem depends on the generating set Σ, but it is well-known that the complexity/decidability status of the word problem does not depend on Σ.

SENS conventions (Definition 1 and Section 4)

For group elements $g, h \in G$ or words $g, h \in \Sigma^*$ we write g^h for the conjugate $h^{-1}gh$ and $[h, g]$ for the commutator $h^{-1}g^{-1}hg$. We call g a d-fold nested commutator, if $d = 0$ or $g = [h_1, h_2]$ for $(d-1)$-fold nested commutators h_1, h_2. A subquotient of a group G is a quotient of a subgroup of G.

Definition 1. *We call a group G with the finite standard generating set Σ* strongly efficiently non-solvable (SENS) *if for every $d \in \mathbb{N}$ there is a collection of $2^{d+1} - 1$ elements $g_{d,v} \in \Sigma^*$ for $v \in \{0,1\}^{\leq d}$ such that*

(a) there is some constant $\mu \in \mathbb{N}$ with $|g_{d,v}| = 2^{\mu d}$ for all $v \in \{0,1\}^d$,

(b) $g_{d,v} = \left[g_{d,v0},\, g_{d,v1}\right]$ for all $v \in \{0,1\}^{<d}$ (here we take the commutator of words),

(c) $g_{d,\varepsilon} \neq 1$ in G.

The group G is called uniformly strongly efficiently non-solvable *if, moreover,*

(d) given $v \in \{0,1\}^d$, a number i encoded in binary with μd bits, and $a \in \Sigma$ one can decide in DLINTIME *whether the i-th letter of $g_{d,v}$ is a.*

Here are some simple observations:

- A strongly efficiently non-solvable group clearly cannot be solvable, so the above terminology makes sense.
- If one can find suitable $g_{d,v}$ of length at most $2^{\mu d}$, then these words can always be padded to length $2^{\mu d}$ thanks to the padding letter 1.
- It suffices to specify $g_{d,v}$ for $v \in \{0,1\}^d$; the other $g_{d,v}$ are then defined by Condition (b).
- We have $|g_{d,v}| = 2^{\mu d + 2(d-|v|)}$ for all $v \in \{0,1\}^{\leq d}$. Thus, all $g_{d,v}$ have length $2^{\mathcal{O}(d)}$.
- Equivalently to Condition (d), we can require that given $v \in \{0,1\}^d$ and a binary encoded number i with μd bits, one can compute the i-th letter of $g_{d,v}$ in DLINTIME.

Henceforth, whenever d is clear, we simply write g_v instead of $g_{d,v}$.

Necessary group context (Section 2)

Richard Thompson introduced three finitely presented groups $F < T < V$ acting on the unit-interval, the unit-circle and the Cantor set, respectively. A standard reference for Thompson's groups is [12]. The group F consists of all homeomorphisms of the unit interval that are piecewise affine, with slopes a power of 2 and dyadic breakpoints. Famously, F has a finite presentation with two generators:

$$F = \langle x_0, x_1 \mid [x_0 x_1^{-1}, x_0^{-1} x_1 x_0], [x_0 x_1^{-1}, x_0^{-2} x_1 x_0^2] \rangle.$$

Very convenient is also the following infinite presentation:

$$F = \langle x_0, x_1, x_2, \dots \mid x_k^{x_i} = x_{k+1} \ (i < k) \rangle.$$

The group F is orderable (so in particular torsion-free), its derived subgroup $[F, F]$ is simple and the center of F is trivial. Let X be a finite set. The free monoid X^* serves as the vertex set of a regular rooted tree with an edge between v and vx for all $v \in X^*$ and all $x \in X$. The group W of automorphisms of this tree naturally acts on the set X of level-1 vertices, and permutes the subtrees hanging from them. Exploiting the bijection $X^+ = X^* \times X$, we thus have an isomorphism

$$\varphi : W \to W \wr \mathrm{Sym}(X) = W^X \rtimes \mathrm{Sym}(X),$$

mapping $g \in W$ to elements $f \in W^X$ and $\pi \in \mathrm{Sym}(X)$ as follows: π is the restriction of g to $X \subseteq X^*$, and f is uniquely defined by $(xv)^g = x^\pi v^{f(x)}$. We always write $g@x$ for $f(x)$ and call it the state (or coordinate) of g at x. If $X = \{0, \dots, k\}$ we write $g = \langle\langle g@0, \dots, g@k \rangle\rangle \pi$. There exist important examples of f.g. self-similar weakly branched groups, notably the Grigorchuk group G, see [17]. It may be described as a self-similar group in the following manner: it is a group generated by $\{a, b, c, d\}$, and acts on the rooted tree X^* for $X = \{0, 1\}$. The action, and therefore the whole group, are defined by the restriction of φ to G's generators: $\varphi(a) = (0, 1)$, $\varphi(b) = \langle\langle a, c \rangle\rangle$, $\varphi(c) = \langle\langle a, d \rangle\rangle$, and $\varphi(d) = \langle\langle 1, b \rangle\rangle$, where we use the notation $(0, 1)$ for the non-trivial element of $\mathrm{Sym}(X)$ (that permutes 0 and 1) and $\langle\langle w_0, w_1 \rangle\rangle$ for a tuple in $G^{\{0,1\}} \cong G \times G$.

The complete formal non-uniform construction (Section 4)

The proof of Theorem 2 essentially follows Barrington's proof that the word problem of finite non-solvable groups is ALOGTIME-hard [3]. The entire proof can be found in the full version [5], where we also state a non-uniform version of Theorem 2. Since the proof for the uniform case is not difficult but tedious, in this sketch we primarily focus on the non-uniform version, which only gives us hardness via (non-uniform) AC^0-reductions instead of DLOGTIME-reductions. Like in Barrington's proof, we start with an ALOGTIME-machine (resp. NC^1-circuit) and construct a family of so-called G-programs. Since we are dealing with finitely generated, but infinite groups, we have to adapt the definition of G-programs slightly. Fix a finite standard generating set Σ of G. A G-program P of length m and input length n is a sequence of instructions $\langle i_j, b_j, c_j \rangle$ for $0 \leq j \leq m - 1$ where $i_j \in [1..n]$ and $b_j, c_j \in \Sigma$. On input of a word $x = x_1 \cdots x_n \in \{0, 1\}^*$, an instruction $\langle i_j, b_j, c_j \rangle$ evaluates to b_j if $x_{i_j} = 1$ and to c_j otherwise. The evaluation of a G-program is the product (in the specified order) of the evaluations of its instructions, and is denoted with $P[x] \in \Sigma^*$. Let M be an ALOGTIME-machine in input normal form [47, Lemma 2.41], i.e., every computation path queries at most one input bit and M halts immediately after the query. For every input size n, the computation tree of M translates immediately into a Boolean circuit of depth $d \in O(\log n)$. Moreover, M can be normalized such that this circuit is a fully balanced binary tree meaning that the gates of the circuit are indexed by the set $\{0, 1\}^{\leq d}$, where $\{0, 1\}^{<d}$ are the inner gates (where ε is the output gate, which counts here as an inner gate)

and $\{0,1\}^d$ are the leaves (input gates). We can assume that all inner gates are nand-gates (where the Boolean function $\mathsf{nand} : \{0,1\}^2 \to \{0,1\}$ is defined by $\mathsf{nand}(0,0) = \mathsf{nand}(0,1) = \mathsf{nand}(1,0) = 1$ and $\mathsf{nand}(1,1) = 0$) and each leaf is labelled by a possibly negated input variable or constant via an input mapping $q_n : \{0,1\}^d \to [1..n] \times \{0,1\} \times \{0,1\}$. The meaning of this mapping is as follows: if the input to the circuit is the bit string $x_1 x_2 \cdots x_n$ and $q_n(v) = \langle i, a, b\rangle$, then the input gate $v \in \{0,1\}^d$ evaluates to a (resp., b) if $x_i = 1$ (resp., $x_i = 0$). The family of circuits obtained this way can be shown to be DLOGTIME-uniform in an even stronger sense than the usual definition (see e.g. [47]). For the sake of a simpler description, we fix the input length n and write C for the n-input circuit of depth $d \in O(\log n)$. W.l.o.g. for every $x \in \{0,1\}^n$, we have $x \in L(M)$ if and only if the output gate of C evaluates to 0 on input x.

Proof. For each gate $v \in \{0,1\}^{\leq d}$ let $g_v = g_{d,v}$ as in Definition 1. We construct two G-programs P_v and P_v^{-1} (both of input length n) such that for every input $x \in \{0,1\}^n$ (x is taken as the input for C, P_v, and P_v^{-1}) we have

$$P_v[x] =_G \begin{cases} g_v & \text{if } v \text{ evaluates to 1,} \\ 1 & \text{if } v \text{ evaluates to 0,} \end{cases} \tag{2}$$

and $P_v^{-1}[x] = P_v[x]^{-1}$ in G. Notice that we have $g_v P_v^{-1}[x] = g_v$ if v evaluates to 0 and $g_v P_v^{-1}[x] = 1$, otherwise. Thus, $g_v P_v^{-1}$ is a G-program for the "negation" of P_v. Moreover, by Equation (2), P_ε evaluates to 1 on input x if and only if the output gate evaluates to 0 which by our assumption was the case if and only if $x \in L$.

The construction of the P_v and P_v^{-1} is straightforward: For an input gate $v \in \{0,1\}^d$ we simply define P_v to be a G-program evaluating to g_v or 1 – in which case it evaluates to which element depends on $q_n(v)$. More precisely, write $g_v = a_1 \cdots a_m$ with $a_i \in \Sigma$. If $q_n(v) = \langle i, a, b\rangle$ for $i \in [1..n]$ and $a, b \in \{0,1\}$, we set $P_v = \langle i, a_1^a, a_1^b\rangle \cdots \langle i, a_m^a, a_m^b\rangle$ and $P_v^{-1} = \langle i, a_m^{-a}, a_m^{-b}\rangle \cdots \langle i, a_1^{-a}, a_1^{-b}\rangle$. For a nand-gate v with inputs from $v0$ and $v1$, we define

$$\begin{aligned} P_v &= g_v[P_{v1}, P_{v0}] = g_v P_{v1}^{-1} P_{v0}^{-1} P_{v1} P_{v0}, \\ P_v^{-1} &= [P_{v0}, P_{v1}] g_v^{-1} = P_{v0}^{-1} P_{v1}^{-1} P_{v1} P_{v1} g_v^{-1}, \end{aligned}$$

where the g_v and g_v^{-1} represent constant G-programs evaluating to g_v and g_v^{-1}, respectively, irrespective of the actual input (such constant G-programs consist of triples of the form $\langle 1, a, a\rangle$ for $a \in \Sigma$). These constant G-programs are defined via the commutator identities $g_v = \left[g_{v0},\, g_{v1}\right]$ for $v \in \{0,1\}^{<d}$ in Definition 1.

Clearly, by induction we have $P_v[x]^{-1} = P_v^{-1}[x]$ in G (for every input x). Let us show that Equation (2) holds: For an input gate $v \in \{0,1\}^d$, Equation (2) holds by definition. Now, let $v \in \{0,1\}^{<d}$. Then, by induction, we have the following equalities in G:

$$\begin{aligned} P_v[x] = g_v[P_{v1}[x], P_{v0}[x]] &= \begin{cases} g_v & \text{if } v0 \text{ or } v1 \text{ evaluates to 0,} \\ g_v[g_{v1},\, g_{v0}] & \text{if } v0 \text{ and } v1 \text{ evaluate to 1,} \end{cases} \\ &= \begin{cases} g_v & \text{if } v \text{ evaluates to 1,} \\ 1 & \text{if } v \text{ evaluates to 0.} \end{cases} \end{aligned}$$

Note that $[g_{v1}, g_{v0}] = [g_{v0}, g_{v1}]^{-1} = g_v^{-1}$ for the last equality. Thus, P_v satisfies Equation (2). For P_v^{-1} the analogous statement can be shown with the same calculation. For a leaf $v \in \{0,1\}^d$, we have $|g_v| \in 2^{\mathcal{O}(d)} = n^{\mathcal{O}(1)}$ by Condition (a) from Definition 1 (recall that $d \in \mathcal{O}(\log n)$). Hence, P_v^{-1} and P_v have polynomial length in n. Finally, also P_ε has polynomial length in n (with the same argument as for g_ε; see the remark after Definition 1).

This gives us a non-uniform AC^0-reduction (more precisely, a projection reduction) of $L(M)$ to $\mathrm{WP}(G)$. In order to obtain a DLOGTIME-reduction, we apply essentially the same construction. However, we need to introduce some padding so that the function mapping an index i encoded in binary to the i-th instruction of P_ε can be computed in DLINTIME. In order to show the last point, we need condition (d) of the uniform SENS definition (Definition 1). □

Direct proof for Thompson's group (Proposition 21)

One can also prove the uniform SENS property for Thompson's group F directly. Recall the infinite presentation $F = \langle x_0, x_1, x_2, \ldots \mid x_k^{x_i} = x_{k+1} \ (i < k)\rangle$.

Proposition 21. Let $g = x_3 x_2^{-1} \in F$ and define c_v for $v \in \{0,1\}^*$ inductively via

$$c_\varepsilon = \varepsilon, \qquad c_{v0} = x_1 c_v, \qquad c_{v1} = x_0^{-1} x_1 c_v.$$

Finally, for $d \in \mathbb{N}$ and $v \in \{0,1\}^{\leq d}$ let $g_{d,v} = g^{c_v}$. Then $g_{d,v} = [g_{d,v0}, g_{d,v1}]$ for all $d \in \mathbb{N}$ and $v \in \{0,1\}^{\leq d}$ and $g_\varepsilon = g \neq 1$ in F. In particular, G is uniformly SENS.

Proof. Obviously, we have $g_\varepsilon = g$ in F. Moreover, since $x_2 \neq x_3$ (which follows directly from the normal form theorem for F; see e.g. [12, Corollary 2.7]), we have $g \neq 1$ in F (Condition (c) of Definition 1). The identity $g = [g, g^{x_0^{-1}}]^{x_1} = [g^{x_1}, g^{x_0^{-1} x_1}]$ is straightforward to check. Thus, we obtain Condition (b):

$$g_{d,v} = g^{c_v} = [g^{x_1 c_v}, g^{x_0^{-1} x_1 c_v}] = [g^{c_{v0}}, g^{c_{v1}}] = [g_{d,v0}, g_{d,v1}].$$

Moreover, since $|g_{d,v}| = |g| + 2|c_v| \leq |g| + 4d$, we can pad with the identity symbol writing $g_{v,d}$ as a word of length $2^{\mu d}$ for some proper constant $\mu \in \mathbb{N}$ in order to meet Condition (a) (be aware that the lengths need to be computed over a finite standard generating set Σ, e.g. $\Sigma = \{1, x_0, x_0^{-1}, x_1, x_1^{-1}\}$). This shows that F is SENS. Condition (d) of Definition 1 is also straightforward to check by introducing some more padding: we slightly change the definition of the c_v by setting $c_\varepsilon = \varepsilon$, $c_{v0} = 1 x_1 c_v$, and $c_{v1} = x_0^{-1} x_1 c_v$ where $1 \in \Sigma$. This new c_v represents the same group element as the old c_v, but now we have $|c_v| = 2|v|$ for all $v \in \{0,1\}^*$ and all letters at even positions are x_1, while letters at odd positions are either 1 or x_0^{-1} (the $(2j+1)$-st letter of c_v depends on the $(|v|-j)$-th bit of v). Notice that we have $|g_{d,v}| = |g| + 2|c_v| = |g| + 4d$. On input of $v \in \{0,1\}^d$, $i \in \mathbb{N}$ (encoded with μd bits), and $a \in \Sigma$, first decide whether $1 \leq i \leq 2d$, or $2d < i \leq 2d + |g|$, or $2d + |g| < i \leq 4d + |g|$, or $i > 4d + |g|$. This clearly can be done in DLINTIME (recall that g is a constant). Now assume that $1 \leq i \leq 2d$ (i.e., i points into the leading c_v^{-1} of $g_{d,v} = c_v^{-1} g c_v$). If i is odd, one accepts if and only if $a = x_1^{-1}$; if i is even, one accepts if and only if the

$i/2$-th bit of v is zero and $a = 1$ or if the $i/2$-th bit of v is one and $a = x_0$. The other cases are similar. □

Direct proof for Grigorchuk's group

Proposition 22. Consider in the Grigorchuk group $G = \langle a, b, c, d\rangle$ the elements

$$x = (abad)^2 \quad \text{and} \quad y = x^b = bababadabac.$$

Define recursively elements $z_v \in \left\{x, y, x^{-1}, y^{-1}\right\}$ for all $v \in \{0,1\}^*$ as follows:

- $z_\varepsilon = x$;
- if z_v is defined, then we define z_{v0} and z_{v1} according to the following table:

z_v	z_{v0}	z_{v1}
x	x^{-1}	y^{-1}
x^{-1}	y^{-1}	x^{-1}
y	y	x
y^{-1}	x	y

For every $d \in \mathbb{N}$ and $v \in \{0,1\}^{\leq d}$ let $g_{d,v} = z_v$ for $|v| = d$ and $g_{d,v} = [g_{d,v0}, g_{d,v1}]$ if $|v| < d$. We then have $g_{d,\varepsilon} \neq 1$ in G. In particular, G is uniformly SENS.

Proof. That $x \neq 1 \neq y$ is easy to check by computing the action of x and y on the third level of the tree. Now the following equations are easy to check in G:

$$\begin{aligned}
[x, y] &= \left\langle\!\left\langle 1, \left\langle\!\left\langle 1, y^{-1}\right\rangle\!\right\rangle\!\right\rangle, \\
[x^{-1}, y^{-1}] &= \left\langle\!\left\langle 1, \left\langle\!\left\langle 1, x\right\rangle\!\right\rangle\!\right\rangle, \\
[y, x] &= \left\langle\!\left\langle 1, \left\langle\!\left\langle 1, y\right\rangle\!\right\rangle\!\right\rangle, \\
[y^{-1}, x^{-1}] &= \left\langle\!\left\langle 1, \left\langle\!\left\langle 1, x^{-1}\right\rangle\!\right\rangle\!\right\rangle.
\end{aligned}$$

Hence, $[z_{v0}, z_{v1}] = \left\langle\!\left\langle 1, \left\langle\!\left\langle 1, z_v\right\rangle\!\right\rangle\!\right\rangle$. The checks are tedious to compute by hand, but easy in the GAP package FR (note that vertices are numbered from 1 in GAP and from 0 here):

```
gap> LoadPackage("fr");
gap> AssignGeneratorVariables(GrigorchukGroup);
gap> x := (a*b*a*d)^2; y := x^b;
gap> Assert(0,Comm(x,y) = VertexElement([2,2],y^-1));
gap> Assert(0,Comm(x^-1,y^-1) = VertexElement([2,2],x));
gap> Assert(0,Comm(y,x) = VertexElement([2,2],y));
gap> Assert(0,Comm(y^-1,x^-1) = VertexElement([2,2],x^-1));
```

We claim that $g_{d,\varepsilon} \neq 1$ in G. The equation $[z_{v0}, z_{v1}] = \left\langle\!\left\langle 1, \left\langle\!\left\langle 1, z_v\right\rangle\!\right\rangle\!\right\rangle$ immediately implies that $g_{d,v}$ acts as z_v on the subtree below vertex $1^{2(d-|v|)}$ and trivially elsewhere. In particular, $g_{d,\varepsilon}$ acts as $z_\varepsilon = x \neq 1$ on the subtree below vertex 1^{2d} and is non-trivial.

With this definition, the $g_{d,v}$ satisfy the definition of a SENS group. Moreover, given some $v \in \{0,1\}^d$, $g_{d,v}$ can be computed in time $\mathcal{O}(d)$ by a deterministic finite state automaton with state set $\left\{x^{\pm 1}, y^{\pm 1}\right\}$. □

References

[1] Ian Agol. The virtual Haken conjecture. Documenta Mathematica, 18:1045–1087, 2013. With an appendix by Ian Agol, Daniel Groves, and Jason Manning.

[2] Sanjeev Arora and Boaz Barak. Computational Complexity - A Modern Approach. Cambridge University Press, 2009. doi:10.1017/CBO9780511804090.

[3] David A. Mix Barrington. Bounded-width polynomial-size branching programs recognize exactly those languages in NC^1 . Journal of Computer and System Sciences, 38(1):150–164, 1989. doi:10.1016/0022-0000(89)90037-8.

[4] David A. Mix Barrington and Denis Thérien. Finite monoids and the fine structure of NC^1 . Journal of the ACM, 35:941–952, 1988. doi:10.1145/48014.63138.

[5] Laurent Bartholdi, Michael Figelius, Markus Lohrey, and Armin Weiß. Groups with ALOGTIME-hard word problems and PSPACE-complete compressed word problems. Technical report, arXiv.org, 2020. arXiv:1909.13781.

[6] Laurent Bartholdi, Rostislav I. Grigorchuk, and Zoran Šuniḱ. Branch groups. In Handbook of algebra, Volume 3, pages 989–1112. Elsevier/North-Holland, Amsterdam, 2003. doi: 10.1016/S1570-7954(03)80078-5.

[7] Laurent Bartholdi and Volodymyr V. Nekrashevych. Iterated monodromy groups of quadratic polynomials. I. Groups, Geometry, and Dynamics, 2(3):309–336, 2008. doi:10.4171/GGD/42.

[8] Martin Beaudry, Pierre McKenzie, Pierre Péladeau, and Denis Thérien. Finite monoids: From word to circuit evaluation. SIAM Journal on Computing, 26(1):138–152, 1997. doi: 10.1137/S0097539793249530.

[9] Richard Beigel and John Gill. Counting classes: Thresholds, parity, mods, and fewness. Theoretical Computer Science, 103(1):3–23, 1992. doi:10.1016/0304-3975(92)90084-S.

[10] William W. Boone. The Word Problem. Annals of Mathematics, 70(2):207–265, 1959. URL: www.jstor.org/stable/1970103.

[11] Daniel P. Bovet, Pierluigi Crescenzi, and Riccardo Silvestri. A uniform approach to define complexity classes. Theoretical Computer Science, 104(2):263–283, 1992. doi:10.1016/ 0304-3975(92)90125-Y.

[12] John W. Cannon, William J. Floyd, and Walter R. Parry. Introductory notes on Richard Thompson's groups. L'Enseignement Mathématique, 42(3):215–256, 1996.

[13] Moses Charikar, Eric Lehman, Ding Liu, Rina Panigrahy, Manoj Prabhakaran, Amit Sahai, and Abhi Shelat. The smallest grammar problem. IEEE Transactions on Information Theory, 51(7):2554–2576, 2005. doi:10.1109/TIT.2005.850116.

[14] Max Dehn. Über unendliche diskontinuierliche Gruppen. Mathematische Annalen, 71(1):116– 144, 1911. doi:10.1007/BF01456932.

[15] Michael R. Garey and David S. Johnson. Computers and Intractability: A Guide to the Theory of NP–completeness. Freeman, 1979.

[16] Max Garzon and Yechezkel Zalcstein. The complexity of Grigorchuk groups with application to cryptography. Theoretical Computer Science, 88(1):83–98, 1991. doi:10.1016/0304-3975(91) 90074-C.

[17] Rostislav I. Grigorchuk. Burnside's problem on periodic groups. Functional Analysis and Its Applications, 14:41–43, 1980. doi:10.1007/BF01078416.

[18] Rostislav I. Grigorchuk and Zoran Šuniḱ. Asymptotic aspects of Schreier graphs and Hanoi Towers groups. C. R. Math. Acad. Sci. Paris, 342(8):545–550, 2006. doi:10.1016/j.crma. 2006.02.001.

[19] Victor S. Guba and Mark V. Sapir. On subgroups of the R. Thompson group F and other diagram groups. Matematicheskii Sbornik, 190(8):3–60, 1999. doi:10.1070/ SM1999v190n08ABEH000419.

[20] Narain Gupta and Saïd Sidki. On the Burnside problem for periodic groups. Mathematische Zeitschrift, 182(3):385–388, 1983. doi:10.1007/BF01179757.

[21] Frédéric Haglund and Daniel T. Wise. Coxeter groups are virtually special. Advances in Mathematics, 224(5):1890–1903, 2010. doi:10.1016/j.aim.2010.01.011.

[22] Ulrich Hertrampf. Über Komplexitätsklassen, die mit Hilfe von k-wertigen Funktionen definiert werden. Habilitationsschrift, Universität Würzburg, 1994.

[23] Ulrich Hertrampf. The shapes of trees. In Proceedings of COCOON 1997, volume 1276 of Lecture Notes in Computer Science, pages 412–421. Springer, 1997. doi:10.1007/BFb0045108.

[24] Ulrich Hertrampf. Algebraic acceptance mechanisms for polynomial time machines. SIGACT News, 31(2):22–33, 2000. doi:10.1145/348210.348215.

[25] Ulrich Hertrampf, Clemens Lautemann, Thomas Schwentick, Heribert Vollmer, and Klaus W. Wagner. On the power of polynomial time bit-reductions. In Proceedings of the Eighth Annual Structure in Complexity Theory Conference, pages 200–207. IEEE Computer Society Press, 1993. doi:10.1109/SCT.1993.336526.

[26] Ulrich Hertrampf, Heribert Vollmer, and Klaus Wagner. On balanced versus unbalanced computation trees. Mathematical Systems Theory, 29(4):411–421, 1996. doi:10.1007/BF01192696.

[27] Derek F. Holt, Markus Lohrey, and Saul Schleimer. Compressed decision problems in hyperbolic groups. In Proceedings of STACS 2019, volume 126 of LIPIcs, pages 37:1–37:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019. URL: http://www.dagstuhl.de/dagpub/ 978-3-95977-100-9.

[28] Derek F. Holt, Sarah Rees, and Claas E. Röver. Groups, Languages and Automata, volume 88 of London Mathematical Society Student Texts. Cambridge University Press, 2017. doi: 10.1017/9781316588246.

[29] Birgit Jenner, Pierre McKenzie, and Denis Thérien. Logspace and logtime leaf languages. Information and Computation, 129(1):21–33, 1996. doi:10.1006/inco.1996.0071.

[30] Howard J. Karloff and Walter L. Ruzzo. The iterated mod problem. Information and Computation, 80(3):193–204, 1989. doi:10.1016/0890-5401(89)90008-4.

[31] Daniel König and Markus Lohrey. Evaluation of circuits over nilpotent and polycyclic groups. Algorithmica, 80(5):1459–1492, 2018. doi:10.1007/s00453-017-0343-z.

[32] Daniel König and Markus Lohrey. Parallel identity testing for skew circuits with big powers and applications. International Journal of Algebra and Computation, 28(6):979–1004, 2018. doi:10.1142/S0218196718500431.

[33] Jörg Lehnert and Pascal Schweitzer. The co-word problem for the Higman-Thompson group is context-free. Bulletin of the London Mathematical Society, 39(2):235–241, February 2007. doi:10.1112/blms/bdl043.

[34] Martin W. Liebeck, Eamonn A. O'Brien, Aner Shalev, and Pham Huu Tiep. The Ore conjecture. Journal of the European Mathematical Society, 12(4):939–1008, 2010. doi:10.4171/JEMS/220.

[35] Yury Lifshits and Markus Lohrey. Querying and embedding compressed texts. In Proceedings of MFCS 2006, volume 4162 of Lecture Notes in Computer Science, pages 681–692. Springer, 2006. doi:10.1007/11821069_59.

[36] Richard J. Lipton and Yechezkel Zalcstein. Word problems solvable in logspace. Journal of the Association for Computing Machinery, 24(3):522–526, 1977. doi:10.1145/322017.322031.

[37] Markus Lohrey. Leaf languages and string compression. Information and Computation, 209(6):951–965, 2011. doi:10.1016/j.ic.2011.01.009.

[38] Markus Lohrey. The Compressed Word Problem for Groups. Springer Briefs in Mathematics. Springer, 2014. doi:10.1007/978-1-4939-0748-9.

[39] Volodymyr Nekrashevych. Self-similar groups, volume 117 of Mathematical Surveys and Monographs. American Mathematical Society, Providence, RI, 2005. doi:10.1090/surv/117.

[40] Piotr S. Novikov. On the algorithmic unsolvability of the word problem in group theory. Trudy Mat. Inst. Steklov, pages 1–143, 1955. In Russian. URL: http://mi.mathnet.ru/eng/tm1180.

[41] David Robinson. Parallel Algorithms for Group Word Problems. PhD thesis, University of California, San Diego, 1993.

[42] Joseph J. Rotman. An Introduction to the Theory of Groups (fourth edition). Springer, 1995.

[43] Amir Shpilka and Amir Yehudayoff. Arithmetic circuits: A survey of recent results and open questions. Foundations and Trends in Theoretical Computer Science, 5(3-4):207–388, 2010. doi:10.1561/0400000039.

[44] Hans-Ulrich Simon. Word problems for groups and contextfree recognition. In Proceedings of FCT 1979, pages 417–422. Akademie-Verlag, 1979.

[45] Roman Smolensky. Algebraic methods in the theory of lower bounds for boolean circuit complexity. In Proceedings of STOC 1987, pages 77–82. ACM, 1987. doi:10.1145/28395. 28404.

[46] Jacques Tits. Free subgroups in linear groups. Journal of Algebra, 20(2):250–270, 1972. doi:10.1016/0021-8693(72)90058-0.

[47] Heribert Vollmer. Introduction to Circuit Complexity. Springer, Berlin, 1999. doi:10.1007/ 978-3-662-03927-4.

[48] Stephan Waack. The parallel complexity of some constructions in combinatorial group theory. Journal of Information Processing and Cybernetics EIK, 26:265–281, 1990. doi: 10.1007/BFb0029647.

[49] Jan Philipp Wächter and Armin Weiß. An automaton group with pspace-complete word problem. In Proceedings of STACS 2020, volume 154 of LIPIcs, pages 6:1–6:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020. URL: https://www.dagstuhl.de/dagpub/ 978-3-95977-140-5.

[50] John S. Wilson. Embedding theorems for residually finite groups. Mathematische Zeitschrift, 174(2):149–157, 1980. doi:10.1007/BF01293535.

[51] Daniel T. Wise. Research announcement: the structure of groups with a quasiconvex hierarchy. Electronic Research Announcements in Mathematical Sciences, 16:44–55, 2009. URL: http://aimsciences.org//article/id/8d3fc128-8d02-4fee-af67-38001ca1d0ac.